Data Mining Assignment 1 — Explained From Scratch

VU Amsterdam | ODI-2026 Dataset | Plain Language Guide

> **Who is this for?** This document explains everything we did in this assignment — every task, every decision, every result — in plain language. No prior knowledge of data mining, machine learning, or statistics is assumed. If you have never heard of a Decision Tree or a KNN algorithm before, this document will explain what they are, why we used them, and what the results actually mean.

PART 0 — What Is This Assignment About?

The big picture

A data mining project is about finding useful patterns in data. Think of it like being a detective: you have a pile of raw information (a dataset), and your job is to clean it up, explore it, and then build models that can learn from it and make predictions.

This assignment follows a standard process called **CRISP-DM** (Cross-Industry Standard Process for Data Mining). It has six steps:

1. **Business Understanding** — what question are we trying to answer?
2. **Data Understanding** — what data do we have, and what does it look like?
3. **Data Preparation** — clean and prepare the data for modelling
4. **Modelling** — train algorithms that learn patterns from the data
5. **Evaluation** — measure how good those predictions are
6. **Deployment** — (not covered in this assignment)

We worked through steps 2–5 in this assignment.

The dataset

At the start of Lecture 1, all 156 students in the Data Mining Techniques course at VU Amsterdam filled in a survey. The survey asked things like:

- What is your gender?
- What is your date of birth?
- What is your stress level right now (0–100)?
- How many hours per week do you do sports?
- What time did you go to bed last night?
- Have you taken courses in machine learning, statistics, databases, or information retrieval?
- How many students do you estimate are in this room?
- A random number between 0 and 100

The result was a file called **ODI-2026.csv** — 156 rows (one per student) and 16 columns (one per question). This is the dataset we worked with throughout the entire assignment.

What were we trying to predict?

We had two prediction goals:

1. **Classification task (Task 2):** Can we predict whether a student is a *late sleeper* (went to bed at midnight or later) based on their other survey answers?
2. **Regression task (Task 4):** Can we predict how *stressed* a student is (a number from 0 to 100) based on their other survey answers?

The difference: - **Classification** = predicting a category (yes/no, true/false, cat/dog) - **Regression** = predicting a number (47.5, 82.0, 13.2)

PART 1 — Task 1A: Exploratory Data Analysis (EDA)

What is EDA and why do we do it?

Before you do anything with data, you need to understand what you're working with. EDA is exactly that — exploring the data like you would explore a new city before deciding where to go. You look around, note what's interesting, spot what's broken, and figure out what needs attention.

Think of it like receiving a box of survey forms. Before doing any analysis, you'd flip through them and notice: "Most people filled this in properly, but a few wrote completely nonsensical answers. Some left sections blank. The handwriting on a few pages is unreadable." That's EDA.

What we found

Basic facts: - **156 students**, **16 columns** of information per student - The data types were all wrong — most columns were stored as plain text (called "objects") even when they should have been numbers or dates. This happened because students gave inconsistent answers (e.g. some wrote "yes", others wrote "1", others wrote "ja" — all meaning the same thing, but the computer couldn't recognise them as the same)

Missing values: - The survey used the word "unknown" when data was missing — not an empty cell, but the actual word "unknown". This is a problem because Python's data tools don't treat "unknown" as missing — they treat it as a valid answer. We had to manually find and handle these 29 cells.

Interesting findings from the plots:

Stress levels: - The distribution (histogram) showed students were spread across the full range (0–100) - A few extreme values stood out: stress scores of 101 and 300 — clearly errors (the scale only goes to 100)

Sports hours: - One student reported −1 hours of sport per week — physically impossible - A few students wrote things like "5-6 hours" or "1hr" instead of a number

Gender: - 5 categories: male (79), female (69), non-binary, gender fluid, not willing to answer - The values here were actually consistent — one of the few clean columns in the dataset

Student count estimates: - Students were asked to estimate how many people were in the lecture hall - Answers ranged from 65 to 500 — a huge spread, showing that humans are very bad at estimating crowd sizes - Many people rounded to nice numbers like 200 or 250

The most important finding — the correlation heatmap:

A correlation measures how two numbers are related. The correlation value ranges from −1 to +1: - **+1** = as one goes up, the other always goes up (perfect positive link) - **0** = no relationship at all - **−1** = as one goes up, the other always goes down (perfect negative link)

We calculated the correlation between all four numeric columns (stress, sports hours, student count estimate, random number).

Key result: Stress and sports hours had a correlation of **−0.31**. This means students who do more sport tend to report lower stress levels. It's a moderate negative relationship — not a perfect link, but a real pattern worth noting. All other correlations were near zero (no meaningful relationship).

This finding — stress vs. sports = −0.31 — became important later when we tried to predict stress levels.

PART 2 — Task 1B: Data Cleaning

What is data cleaning and why do we need it?

Raw survey data is almost always messy. People answer questions in unexpected ways, make typos, skip questions, or give impossible values. Machine learning algorithms cannot deal with this — they need clean, consistent, numeric data.

Data cleaning is the process of: 1. Removing or correcting impossible/extreme values (called **outliers**) 2. Filling in missing values (called **imputation**)

Think of it like this: if you asked 156 people for their height and one person wrote "500 cm", you'd know that's an error. You'd remove it. And if some people left the height field blank, you'd need a strategy to fill it in.

Step 1: Removing outliers

What is an outlier? A value that is so far outside the expected range that it's almost certainly an error.

What we removed: - Stress values of **101, 101, and 300** — the scale only goes from 0 to 100. These are impossible values, so we removed them. We also removed entries like "Yes" and "9,999,999" in the stress column (clearly someone typed in the wrong field). - Sports hours value of **−1** — you cannot do negative hours of sport. Also removed non-numeric entries like "5-6 hours" and "1hr".

Total after cleaning: stress had 6 missing values, sports had 7 missing values. These needed to be filled in (imputed).

Why didn't we just delete those students' entire rows? Because we only had 156 students to begin with. Throwing away entire rows because of one bad value wastes the rest of that student's valid data. It's better to fill in the missing value intelligently.

Step 2: Imputing missing numeric values

What is imputation? Filling in missing values with an estimated value based on the rest of the data.

We compared two approaches:

Option A: Mean imputation — fill in with the average value - Stress mean = 47.19 - Sports mean = 4.68

Option B: Median imputation — fill in with the middle value (sort all values, take the one in the exact middle) - Stress median = 50.0 - Sports median = 4.0

Which did we choose and why?

We chose **median** for both.

Here's the intuition: imagine 10 people's salaries are: €30k, €32k, €35k, €33k, €31k, €34k, €36k, €38k, €35k, and €1,000,000 (a billionaire).

- The **mean** salary = €128,300 — massively distorted by the billionaire. If you imputed anyone's missing salary with this value, it would be completely wrong.
- The **median** salary = €34,500 — the actual middle value, unaffected by the extreme outlier.

In our dataset, stress had outliers (300, 101, 101 — which we removed, but the distribution was still skewed). The median is more **robust** to this kind of skew. This is also supported by Little & Rubin (2002), a well-cited academic reference on missing data strategies.

Step 3: Imputing missing categorical values

The four course columns (machine learning, information retrieval, statistics, databases) had missing "unknown" values. These are categorical — they contain labels like "yes"/"no", not numbers.

You cannot use mean or median for categorical data. The "mean" of yes/no is meaningless.

Instead, we used **mode imputation** — fill in the most common answer in that column: - Machine learning: most students said **yes** → fill missing with yes - Information retrieval: most students said **no (0)** → fill missing with 0 - Statistics: most students said **yes (mu)** → fill missing with mu - Databases: most students said **yes (ja)** → fill missing with ja

Limitation we acknowledged: Mode imputation reinforces the majority. If the true answer for a missing student was actually "no", we're wrong. But with 156 students and limited information, this is the best we can do without additional data.

PART 3 — Task 1C: Feature Engineering

What is feature engineering?

Machine learning algorithms only understand **numbers**. They cannot understand text, dates, or inconsistent formats.

Feature engineering is the process of transforming raw data into a format that algorithms can actually use — often creating new, more useful columns from the existing messy ones.

Think of it like translating a foreign-language document before feeding it into a translator tool. The tool can only work with the translated version.

What we engineered

Feature 1: Standardise the course columns

The four course columns each used a completely different format to say the same thing (yes or no):

Column	Format used
Machine learning	yes / no (English)
Information retrieval	1 / 0 (numbers)
Statistics	mu / sigma (Greek letters!)
Databases	ja / nee (Dutch)

All four ask the exact same type of question. We converted everything to a consistent binary format: **1 = yes, 0 = no**.

Why? Because algorithms need numeric inputs. A tree or KNN algorithm cannot measure the "distance" between "mu" and "sigma" — but it can compare 1 and 0.

Feature 2: Extract age from birthday

The birthday column contained 153 unique values in all kinds of formats: "27/6/2001", "31.01.97", "5 November", "03-04-1999", and more.

We wrote code to parse as many of these as possible and convert them into a single clean number: **age** (in years as of 2025).

- 115 out of 156 were parsed successfully
- 41 entries were unparseable (e.g. "5 November" with no year — impossible to calculate age from)

- These 41 were filled with the **median age of 24**
- Final age range: 21–36 — realistic for university students

Why age? Age is a meaningful variable that an algorithm can work with. "27/6/2001" is not.

Feature 3: Create a "late sleeper" binary column

The bedtime column had values like "22:00", "03:30", "24:00", "1am", "after midnight", etc. Again, completely inconsistent.

We converted these into a binary column: **late_sleeper** - 1 = went to bed at midnight (00:00) or later - 0 = went to bed before midnight

Result: 102 students (65%) were late sleepers, 54 (35%) were not.

This became our **classification target** in Task 2 — the thing we wanted to predict.

Why this column as the target? Because it's a clean binary (yes/no) outcome derived from a real behaviour (sleep time), and it could plausibly be related to other survey answers like stress, sports habits, and course background.

PART 4 — Task 2A: Classification

What is classification?

Classification is about putting things into categories. A classification algorithm **learns from examples** and then makes predictions.

Simple real-world examples: - Email spam filter: learns from thousands of emails labelled "spam" or "not spam", then predicts whether new emails are spam - Medical diagnosis: learns from patient records labelled "cancer" or "no cancer", then predicts for new patients

Our classification task: **can we predict whether a student is a late sleeper based on their survey answers?**

Setting up the problem

Target (what we want to predict): late_sleeper (1 = yes, 0 = no)

Features (the inputs we give the algorithm): - 4 course columns (binary 0/1) - Stress level (number) - Sports hours (number) - Age (number) - Gender (5 binary columns, one per gender category — see "one-hot encoding" below) - **Total: 12 input features**

What is one-hot encoding? Algorithms can't process a text label like "male" or "female" directly. So we converted the gender column into 5 separate binary (0/1) columns — one for each gender category. Each student gets a 1 in their gender column and 0 in all others. This is called one-hot encoding.

The train/test split:

We had 156 students. We split them into two groups *before* training anything:

Group	Size	Purpose
Training set	124 students (80%)	The algorithm studies these — it sees both their answers AND the correct label
Test set	32 students (20%)	The algorithm is tested on these — it only sees the answers, not the labels

Why split? Because if you train and test on the same data, the algorithm has already "seen the answers" — like giving a student the exam questions in advance. The score would look great but tell you nothing about real performance.

We also used **stratified splitting** — this ensures the 65/35 (late sleeper/not) ratio is maintained in both the training and test sets. Without this, you might accidentally put all the "not late sleepers" in the training set.

The baseline:

Before building any model, we established a **baseline** — the simplest possible prediction strategy. Our baseline was: *always predict "late sleeper" for every single student*.

Why? Because 65% of students ARE late sleepers. If you always predict the majority class, you're right 65% of the time without learning anything.

Any model we build **must beat 65.6% accuracy** to be useful. Otherwise we could have just said "late sleeper" every time without doing any work.

Result: Baseline accuracy = 65.6%

Algorithm 1: Decision Tree Classifier

What is a Decision Tree?

A Decision Tree is exactly what it sounds like — a tree of yes/no decisions. Think of it like a flowchart:

```
Does the student do more than 5 hours of sport per week?
├── YES → Is their stress level above 50?
│         ├── YES → Predict: not late sleeper
│         └── NO  → Predict: late sleeper
└── NO  → Did they take a machine learning course?
          ├── YES → Predict: late sleeper
          └── NO  → Predict: not late sleeper
```

The algorithm automatically figures out which questions to ask and in which order by finding the split that best separates late sleepers from non-late sleepers at each step. This is called **information gain** — it measures how much each question reduces uncertainty.

The key setting: max_depth

`max_depth` controls how many questions the tree is allowed to ask: - `max_depth=1` → ask 1 question, make a prediction. Very simple. - `max_depth=10` → ask up to 10 questions. More complex, potentially more accurate — but also more likely to **overfit**.

What is overfitting? When a model learns the training data *too well* — including all its quirks and noise — and then performs badly on new data. It's like a student who memorises last year's exam answers word-for-word instead of actually understanding the subject. They'd ace last year's exam but fail a new one.

Hyperparameter tuning:

We tested every depth from 1 to 10 and measured accuracy on the test set:

max_depth	Accuracy
1	65.6%
2	65.6%
3	65.6%
4	50.0%
5–10	53–59%

Best depth: **1**, with accuracy **65.6%**.

What this means: The best Decision Tree accuracy (65.6%) is exactly equal to the baseline (65.6%). The tree is not learning any real patterns — it's just predicting "late sleeper" for every student (the same thing the baseline does).

Why did this happen? 1. **Small dataset** — with only 124 training students and 12 features, there isn't enough data for the tree to find reliable patterns 2. **Weak signal** — the features we have (stress, sports, courses, gender, age) don't strongly predict sleep time. The correlation analysis in Task 1A showed this: none of our features had a strong relationship with late_sleeper behaviour.

Algorithm 2: KNN Classifier (K-Nearest Neighbours)

What is KNN?

KNN is a completely different approach. Instead of learning rules, it simply **remembers all the training data** and makes predictions by looking at similarity.

When it needs to predict whether a new student is a late sleeper: 1. It calculates how "similar" this student is to every student in the training set 2. It finds the **k most similar students** (the "nearest neighbours") 3. It takes a **majority vote** among those k students 4. It predicts whichever label the majority had

Example with k=3: The 3 most similar training students are: - Student A: late sleeper ✓ - Student B: late sleeper ✓ - Student C: not late sleeper ✗

Majority vote: 2 late sleepers vs 1 not → **predict: late sleeper**

How is "similar" measured?

We measure the **distance** between students in "feature space". Think of each student as a point in a multi-dimensional space — one dimension for stress, one for sports, one for age, etc. Students close together in this space are similar.

Important: Feature normalisation

There's a problem: stress ranges from 0–100, but course columns are 0 or 1. Without normalisation, the stress dimension would completely dominate distance calculations — a stress difference of 30 would look enormous compared to a course difference of 1.

We used **StandardScaler** to normalise all features to the same scale before calculating distances. This ensures every feature contributes equally.

Critical rule: we **only fit the scaler on training data**, never on test data. Why? Because the test set is supposed to simulate new, unseen students. If we included test data in the normalisation, we'd be secretly leaking information about the test set into the model — called **data leakage**. It would make results look better than they actually are.

The key setting: k

k	Accuracy	Note
1	53.1%	Too sensitive — one weird neighbour ruins it
2	40.6%	Ties in binary vote — terrible
5	**71.9%**	Best result
10	68.8%	Averaging too many → less precise
20	62.5%	Almost back to baseline

Best k: **5**, with accuracy **71.9%**.

Why is k=2 terrible? With 2 neighbours and a binary (yes/no) target, you can get a 1-1 tie. The algorithm doesn't know what to predict and falls back to a default behaviour. This is why you should always use odd k values for binary classification.

Why does accuracy drop at high k? With k=20, you're consulting 20 nearest neighbours — including some who aren't that similar at all. The prediction becomes the average of too many diverse students, pulling towards the overall mean rather than reflecting local patterns.

Final classification results

To measure performance, we used two metrics:

Accuracy — the percentage of test students correctly predicted. Simple, but misleading with imbalanced classes.

Macro F1-score — a more honest metric for imbalanced data. It measures how well the model predicts *both* classes (late sleeper AND not late sleeper), averaging them equally. A model that just always predicts "late sleeper" will have a terrible F1-score even if its accuracy looks decent.

Model	Accuracy	Macro F1-score
Baseline (always predict majority)	65.6%	0.396
Decision Tree (depth=1)	65.6%	**0.396**
KNN (k=5)	**71.9%**	**0.640**

What this tells us: - Decision Tree = identical to baseline. It learned nothing. F1 = 0.396 because it never predicted "not late sleeper" at all. - KNN clearly outperforms both baseline and Decision Tree. F1 = 0.640 means it's genuinely predicting both classes with some success. - **Winner: KNN with k=5**

Why did KNN beat the Decision Tree? KNN is more flexible — it doesn't need to build explicit rules. It finds local patterns based on similar students. With a small, noisy dataset, this instance-based approach works better than rule-based splitting.

PART 5 — Task 4: Numerical Prediction (Regression)

What is regression?

Regression is the numerical cousin of classification. Instead of predicting a category (yes/no), you predict a **number**.

Real-world examples: - House price prediction: learn from past house sales, predict the price of a new house - Weather forecasting: learn from historical data, predict tomorrow's temperature (a number) - Our task: learn from survey answers, predict a student's **stress level** (0–100)

The key difference from classification: - Classification: "Is this student a late sleeper?" → Yes or No - Regression: "What is this student's stress level?" → 47.3 or 82.1 or 15.0

You can never be exactly right in regression — predictions are always somewhat off. So instead of measuring accuracy, we measure **how far off** the predictions are.

Setting up the problem

Target (what we want to predict): stress level (number, 0–100)

Features: same 12 as Task 2, except: - Stress removed from features (you can't use the target as its own predictor) - Late_sleeper added as a feature (it was the target in Task 2, now it's useful input)

Baseline: Instead of always predicting the majority class, the regression baseline is: *always predict the average stress level* (47.1). This is the simplest possible strategy — no matter who the student is, guess 47.1.

Any model must **beat this baseline** to be useful.

How we measure performance

MSE (Mean Squared Error) — for each student, calculate the error (real stress − predicted stress), square it, then average all the squared errors.

Example: If a student's real stress is 70 and we predicted 50, the error is 20. Squared = 400.

The squaring matters: an error of 20 contributes 400 to MSE, but an error of 40 contributes 1600 — four times as much. MSE **punishes large errors heavily**. A few very wrong predictions will dramatically increase MSE.

MAE (Mean Absolute Error) — same idea but just take the absolute value (ignore the sign), then average.

An error of 20 contributes 20. An error of 40 contributes 40. MAE treats all errors proportionally — a 40-unit error is exactly twice as bad as a 20-unit error.

RMSE (Root Mean Squared Error) — the square root of MSE. Brings the number back to the same units as the original (stress points), making it easier to interpret.

Baseline results: - MSE = 783.27 - MAE = 24.09 - RMSE = 27.99 → on average, the naive "always predict 47.1" strategy is off by about 28 stress points

Algorithm 1: Decision Tree Regressor

Works the same way as the classifier — asks yes/no questions about the features. The difference is what it predicts at the end of each path: instead of a class label, it predicts the **average stress level** of all training students who ended up in that group.

Hyperparameter tuning (max_depth 1–10):

max_depth	MSE	MAE
1	918.24	26.30
2	1026.03	27.36
3	955.99	26.17
...	...	...
10	1759.88	33.19

Baseline for comparison: MSE=783.27, MAE=24.09

Best depth: **1** with MSE=918.24, MAE=26.30.

Critical finding: every single depth performs worse than the baseline. The Decision Tree Regressor cannot beat simply predicting the mean stress value for every student.

Why? At depth=1, the tree makes exactly one split — it divides students into two groups and predicts one average stress value per group. This produces only two possible predictions

(roughly 42 and 60). Real stress values range from 0 to 100. Two fixed predictions cannot capture this complexity.

At greater depths, the tree overfits the training data and does even worse on the test set.

Algorithm 2: KNN Regressor

Same idea as KNN Classifier — find the k most similar students in training data. The difference: instead of a majority vote, it predicts the **average stress level** of those k neighbours.

Hyperparameter tuning (k 1–20):

k	MSE	MAE
1	1492.09	31.12
5	882.49	24.36
10	843.40	25.07
17	734.87	23.60
20	**692.33**	**22.96**

Baseline for comparison: MSE=783.27, MAE=24.09

Best k: **20** with MSE=692.33, MAE=22.96. This **beats the baseline**.

Why does higher k work better for regression than for classification? Because stress is a very noisy measurement — two students with identical survey answers might have very different stress levels due to personal factors outside the survey. By averaging 20 neighbours, you smooth out individual noise and get a more stable estimate.

Note: at k=1, you're predicting the stress of the single most similar student. If that one student happened to be unusually stressed that day, your prediction is thrown off completely.

Final regression results

Model	MSE	MAE	RMSE
Baseline (predict mean = 47.1)	783.27	24.09	27.99
Decision Tree (depth=1)	918.24	26.30	30.30
KNN Regressor (k=20)	**692.33**	**22.96**	**26.31**

Winner: KNN Regressor with k=20

Even the winner (KNN, RMSE=26.31) is only slightly better than the baseline (RMSE=27.99). On a 0–100 stress scale, the best model is still wrong by about 26 stress points on average. That's not a great prediction.

Why is regression so hard in this dataset?

This is the most important finding to understand. Think about it from a human perspective: if someone gave you a student's answers to these questions:

- Did you take a databases course? Yes
- Do you sport 4 hours a week? Yes
- Are you 24 years old? Yes
- Are you a late sleeper? Yes

Could you accurately predict their stress level? Almost certainly not. Stress is a complex personal experience driven by relationships, deadlines, family situations, sleep quality, finances, and dozens of other things that aren't in this survey.

The correlation analysis in Task 1A confirmed this: the strongest predictor of stress (sports hours) only had a −0.31 correlation. Everything else was near zero. The models have very little signal to learn from.

This is not a failure of the models — it's a reflection of the data. And this is exactly the kind of honest finding that a good data mining report should include.

PART 6 — Task 5A: Characteristics of Evaluation Metrics

When should you use MSE vs MAE?

They measure different things:

MSE heavily punishes large errors. If one prediction is off by 50 stress points, that contributes 2500 to MSE. Compare to an error of 5 points, which contributes only 25. The large error has 100× the impact even though it's only 10× the size.

When to use MSE: When large errors are disproportionately bad. Example: predicting the required dosage of medication. Being off by 50mg might be fatal; being off by 5mg twice is much less dangerous. You want to heavily penalise the rare catastrophic mistake.

MAE treats all errors proportionally. An error of 50 is exactly 10× worse than an error of 5. No squaring, no extra penalty.

When to use MAE: When all errors are equally bad per unit. Example: predicting delivery times. Being 10 minutes late once is just as bad as being 5 minutes late twice. Simple proportional penalty.

When do MSE and MAE give identical model rankings?

If every prediction error is exactly the same size for both models being compared, MSE and MAE will rank them identically. More practically: if two models have the exact same distribution of errors (just in different order), both metrics give the same verdict. In practice, MSE and MAE often agree on which model is better — but they can disagree when one model makes mostly small errors with occasional very large ones (which MSE penalises more heavily).

PART 7 — Task 5B: Impact of Evaluation Metrics

Comparing MSE and MAE in practice

Model	MSE	MAE	RMSE	Gap (RMSE − MAE)
Baseline (predict mean)	783.27	24.09	27.99	3.90 stress points
Decision Tree (depth=1)	918.24	26.30	30.30	4.00 stress points
KNN Regressor (k=20)	**692.33**	**22.96**	**26.31**	**3.35 stress points**

Both metrics agree: KNN is best, Decision Tree is worst.

What the "Gap" tells us

The gap (RMSE − MAE) measures how much large individual errors inflate the score. Here's why: - If all errors were the same size, RMSE = MAE (because squaring identical values doesn't change their relative size) - When some errors are much larger than others, RMSE grows faster than MAE because of the squaring - A bigger gap = more large individual mistakes in that model

Results: - **KNN gap = 3.35** — the smallest gap. KNN's errors are more evenly distributed. It doesn't make many huge mistakes. - **Decision Tree gap = 4.00** — the largest gap. The rigid two-prediction structure (depth=1) means some students get very wrong predictions (e.g. a student with stress=90 who gets predicted 42). - **Baseline gap = 3.90** — in between.

Interpretation: Under MAE alone, KNN (22.96) is only slightly better than the baseline (24.09). But under MSE, KNN looks considerably better relative to the Decision Tree — because KNN avoids the large errors that MSE punishes so heavily.

PART 8 — Summary: What Did We Actually Find?

The honest summary

Task	Finding
1A: EDA	Dataset is messy, inconsistent, and multi-lingual. Only meaningful relationship: stress ↔ sports (−0.31).
1B: Cleaning	Removed 4 impossible values; imputed 6+7 numeric missing (median) and 29 categorical missing (mode).
1C: Features	Created 3 new features: binary course columns, age from birthday, late_sleeper from bedtime.
2A: Classification	KNN (k=5) beats baseline: 71.9% accuracy, F1=0.640. Decision Tree = identical to baseline — learned nothing.
4: Regression	KNN (k=20) barely beats baseline: RMSE=26.31 vs 27.99. Decision Tree is worse than baseline.
5: Metrics	Both MSE and MAE agree: KNN > Baseline > Decision Tree. KNN has smallest error gap, fewest large mistakes.

Why both models struggled

The fundamental problem is the **dataset itself**, not the algorithms: - **Only 156 students** — tiny for machine learning. Algorithms need much more data to find reliable patterns. - **Weak features** — survey questions about courses, sports, and demographics are poor predictors of sleep time and stress levels. These are complex, personal outcomes driven by many factors not captured in this survey.

This is a completely valid academic finding. The assignment rewards honest interpretation — explaining *why* models fail is just as valuable as reporting models that succeed.

Classification vs Regression — key differences observed

	Classification (Task 2)	Regression (Task 4)
Target	Late sleeper: yes/no	Stress: 0–100
Metric	Accuracy + Macro F1	MSE + MAE
Decision Tree	= baseline (no learning)	Worse than baseline
KNN	Beats baseline (+6%)	Barely beats baseline (+6% RMSE reduction)
Why difficult	Small data, weak signal	Stress is inherently complex and personal

---

PART 9 — Key Vocabulary Reference

Term	Plain Language Meaning
Dataset	A table of data — rows are observations (students), columns are variables (survey questions)
Feature	An input variable used to make predictions (e.g. sports hours, age)
Target	The variable you're trying to predict (e.g. late_sleeper, stress)
EDA	Exploratory Data Analysis — exploring the data before modelling
Outlier	A value so extreme it's almost certainly an error (e.g. stress = 300)
Imputation	Filling in missing values with estimated ones
Mean	Average (sum divided by count)
Median	The middle value when sorted — robust to extreme outliers
Mode	The most frequently occurring value
Feature engineering	Creating new, more useful columns from existing messy ones
One-hot encoding	Converting a category column into multiple binary (0/1) columns
Binary	Something with exactly two possible values (0/1, yes/no, true/false)
Classification	Predicting a category (yes/no)
Regression	Predicting a number

Term	Plain Language Meaning
Train/test split	Dividing data so the model learns on one part and is tested on another
Baseline	The simplest possible strategy — any model must beat this to be useful
Overfitting	Model memorises training data too well; performs badly on new data
Underfitting	Model is too simple to learn any patterns
Accuracy	Percentage of test cases correctly predicted
F1-score	Performance metric that balances precision and recall — better for imbalanced data
Macro F1	F1 averaged equally across all classes — fair for imbalanced datasets
Decision Tree	Algorithm that makes predictions using a flowchart of yes/no questions
Information gain	Measure of how much a question reduces uncertainty — used to build decision trees
max_depth	How many questions a decision tree is allowed to ask
KNN	K-Nearest Neighbours — classifies/predicts based on the k most similar training examples
k	The number of neighbours KNN consults when making a prediction
Hyperparameter	A setting of an algorithm that you tune manually (e.g. depth, k)
Hyperparameter tuning	Testing different values of a setting to find the best one
StandardScaler	Tool that normalises all features to the same scale (mean=0, std=1)
Normalisation	Rescaling features so they are all on the same scale
Data leakage	Accidentally letting test data influence the model — invalidates results

Term	Plain Language Meaning
MSE	Mean Squared Error — measures average squared prediction error; penalises large mistakes heavily
MAE	Mean Absolute Error — measures average absolute prediction error; treats all errors proportionally
RMSE	Root Mean Squared Error — square root of MSE, in the same units as the target
Correlation	Statistical measure of the linear relationship between two variables (−1 to +1)
Stratified split	Train/test split that maintains the class ratio in both sets
CRISP-DM	Standard process model for data mining projects (6 steps from business understanding to deployment)
Signal	The useful information in a feature that genuinely predicts the target
Noise	Random variation in data that has nothing to do with the true underlying pattern

---

Document generated from assignment1.ipynb | VU Amsterdam Data Mining Techniques 2026